

Cloud Storage & CDN Pipeline

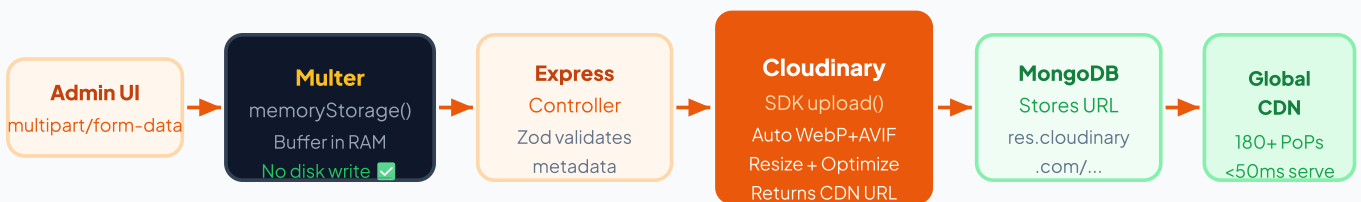
Cloudinary managed media pipeline, Multer in-memory buffer processing, automatic WebP/AVIF format conversion, ephemeral disk hygiene, and global CDN delivery.

☁ Storage: Cloudinary Managed CDN 📁 Upload: Multer Memory Buffer 🖼️ Auto-Format: WebP + AVIF

1. Media Upload Pipeline Architecture

📁 IMAGE UPLOAD LIFECYCLE — FROM BROWSER TO GLOBAL CDN

→ Student browser receives optimized image



🏗️ ARCHITECTURE DECISION: CLOUDINARY MANAGED CDN VS AWS S3 + CLOUDFRONT

✅ Cloudinary (Our Choice)

- ✅ Auto WebP/AVIF format conversion
- ✅ Built-in CDN — 180+ global PoPs
- ✅ Single SDK call → done
- ✅ Automatic resizing & quality optimization
- ✅ Generous free tier (25GB bandwidth)
- ⚡ Upload-to-CDN in 1 API call, <2s

❌ AWS S3 + CloudFront

- ❌ Manual format conversion (Lambda@Edge)
- ❌ Requires configuring CloudFront distribution
- ❌ Separate IAM roles, bucket policies, ACLs
- ❌ Cache invalidation costs money
- ❌ Minimum 3–4 AWS services required
- 🕒 Setup time: 8–16 hours of DevOps work

2. Technology Decision Framework

1. Cloudinary Managed CDN vs AWS S3 + CloudFront

STORAGE ARCHITECTURE

WHAT IS IT?

A cloud media management platform that handles file upload, automatic image format optimization (WebP/AVIF), global CDN delivery, and URL-based transformation in a single SDK.

HOW DOES IT WORK & PROBLEM SOLVED?

Admin uploads a PNG university banner (5MB). Cloudinary receives the buffer stream, automatically converts it to WebP (saving 40% file size), creates multiple responsive size variants, and distributes the optimized image across 180+ global CDN edge nodes. Students receive the nearest-PoP version in under 50ms.

WHAT ALTERNATIVES EXISTED?

AWS S3 + CloudFront: Requires configuring IAM roles, bucket policies, CloudFront distributions, Lambda@Edge for format conversion — 8–16 hours of DevOps setup and ongoing maintenance.

WHY DID WE CHOOSE THIS APPROACH?

Upload-to-global-CDN in one API call. No bucket configuration, no IAM policy debugging, no CloudFront distribution management. Pure business velocity.

2. Multer Memory Buffer vs Disk Storage

SERVER HYGIENE

WHAT IS IT?

Multer middleware configured with `memoryStorage()` mode that holds uploaded file bytes in server RAM as a Buffer, never writing to disk.

HOW DOES IT WORK & PROBLEM SOLVED?

With `diskStorage()`, uploaded files persist as temporary files on the server's filesystem. In ephemeral containerized environments (Render, Railway, Fly.io), these containers restart and the disk is wiped, losing the uploaded files. Worse, if `fs.unlinkSync()` is forgotten, temp files accumulate until the disk fills up, crashing the server.

WHAT ALTERNATIVES EXISTED?

`diskStorage()` + `fs.unlinkSync()`: Works but requires meticulous cleanup code in every error path. One unhandled exception = disk accumulation.

WHY DID WE CHOOSE THIS APPROACH?

Zero disk footprint by design: files live only in RAM for the duration of the upload, then are garbage-collected automatically. Container-native and zero-cleanup-code required.

💡 Senior Interview Defense: "How do you handle media uploads in production?"

"In UniCoach, file uploads use Multer with memoryStorage — images are held in RAM as Buffers and streamed directly to Cloudinary via their SDK, never touching disk. Cloudinary automatically converts uploads to WebP/AVIF, creates responsive size variants, and serves them globally from their 180-PoP CDN. The Cloudinary URL is the only artifact persisted to MongoDB. This is zero-disk, zero-cleanup, and CDN-ready in a single API call — versus AWS S3 + CloudFront which requires 8+ hours of DevOps configuration."